COMP0005 – ALGORITHMS

Week 5 – 11 Feb 2025

Prof Licia Capra

Dept of Computer Science

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Exercises
  - Quicksort
  - Heapsort

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Exercises
  - Quicksort
  - Heapsort

WEEK PREVIEW

- Topic 3 – Search Algorithms
  - 3.1 Symbol Table ADT
  - 3.2 & 3.2b Binary Search Tree (inc. advanced operations)
- Self-practice: Symbol Tables and BSTs
- Lab session: TA Lab – Week 5 (& 6)

No [lecture / labs / office hours] during reading week

Week 6 is the w/c 24 Feb 2025

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Exercises
  - Quicksort
  - Heapsort

MUDDIEST POINT FEEDBACK

Question	Priority	Heat 10/10
Exercise 2 of the TA Lab week 4 Posted by Anonymous at Wednesday, 5 February 2025, 10:47 AM (1 day ago)	0 👍 👎	0 👍
The concept of binary heaps and how they work compared to complete trees Posted by Anonymous at Tuesday, 4 February 2025, 8:02 PM (1 day 14 hours ago)	0 👍 👎	0 👍
How do we do merge sort without copying over the elements to an auxiliary array? I know there is some technique of alternating between the array and the aux array but I don't understand how it actually works. Posted by Anonymous at Tuesday, 4 February 2025, 5:10 PM (1 day 17 hours ago)	0 👍 👎	0 👍
Can you explain the difference between best, worst and average case vs big O, big theta, and big omega? Can you give examples of when it is being used Posted by Anonymous at Tuesday, 4 February 2025, 4:40 PM (1 day 18 hours ago)	0 👍 👎	0 👍

MUDDIEST POINT FEEDBACK

- Complete Binary Tree vs Binary Heap
  - Structure vs (Structure & Content)
  - Binary Heap → Complete Binary Tree
  - Complete Binary Tree $\not\rightarrow$ Binary Heap

MUDDIEST POINT FEEDBACK

Question	Priority	Heat 10/10
Exercise 2 of the TA Lab week 4 Posted by Anonymous at Wednesday, 5 February 2025, 10:47 AM (1 day ago)	0 👍 👎	0 👍
The concept of binary heaps and how they work compared to complete trees Posted by Anonymous at Tuesday, 4 February 2025, 8:02 PM (1 day 14 hours ago)	0 👍 👎	0 👍
How do we do merge sort without copying over the elements to an auxiliary array? I know there is some technique of alternating between the array and the aux array but I don't understand how it actually works. Posted by Anonymous at Tuesday, 4 February 2025, 5:10 PM (1 day 17 hours ago)	0 👍 👎	0 👍
Can you explain the difference between best, worst and average case vs big O, big theta, and big omega? Can you give examples of when it is being used Posted by Anonymous at Tuesday, 4 February 2025, 4:40 PM (1 day 18 hours ago)	0 👍 👎	0 👍

MUDDIEST POINT FEEDBACK

- Merge Sort without copy-over

```
sort(P1[], P2[], lo, hi):
  if (hi <= lo) return;

  mid = lo + (hi - lo) / 2;

  sort(P1, P2, lo, mid);
  sort(P1, P2, mid+1, hi);
  merge(P1, P2, lo, mid, hi);
```

```
merge(P1[], P2[], lo, mid, hi):
  for (k = lo; k <= hi; k++)
    P2[k] = P1[k];

  i = lo;
  j = mid+1;

  for (k = lo; k <= hi; k++)
    if (i > mid)              P1[k] = P2[j]; j++;
    else if (j > hi)          P1[k] = P2[i]; i++;
    else if (P2[j] < P2[i])   P1[k] = P2[j]; j++;
    else                      P1[k] = P2[i]; i++;
```

P1: input & output

P2: working array

```
sort(P1[], P2[], lo, hi):
  if (hi <= lo) return;

  mid = lo + (hi - lo) / 2;

  sort(P2, P1, lo, mid);
  sort(P2, P1, mid+1, hi);
  merge(P1, P2, lo, mid, hi);
```

```
merge(P1[], P2[], lo, mid, hi):
  for (k = lo; k <= hi; k++)
    P2[k] = P1[k];

  i = lo;
  j = mid+1;

  for (k = lo; k <= hi; k++)
    if (i > mid)              P1[k] = P2[j]; j++;
    else if (j > hi)          P1[k] = P2[i]; i++;
    else if (P2[j] < P2[i])   P1[k] = P2[j]; j++;
    else                      P1[k] = P2[i]; i++;
```

P1: output

P2: input & working array

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Exercises
  - Quicksort
  - Heapsort

EXERCISES – QUICKSORT

2022/23 Main Exam – Question 1(a)

Suppose you have the following array `a[]` of 8 elements (indexed from 0 to 7) that you need to sort using the Quicksort recursive algorithm (assume the array has already been shuffled):

a[]	7	1	16	3	4	12	5	8

- Illustrate the content of the whole array `a[]` after completing the first partitioning step (i.e., after returning from a call to `partition(a[],0,7)`). Use as pivot the left-most element in the (sub)array you are working on.
- Illustrate the content of the whole array `a[]` after completing the second partitioning step. Use as pivot the left-most element in the (sub)array you are working on.

EXERCISES – QUICKSORT

2022/23 Main Exam – Question 1(a)

Suppose you have the following array `a[]` of 8 elements (indexed from 0 to 7) that you need to sort using the Quicksort recursive algorithm (assume the array has already been shuffled):

a[]	7	1	16	3	4	12	5	8

- Illustrate the content of the whole array `a[]` after completing the first partitioning step (i.e., after returning from a call to `partition(a[],0,7)`). Use as pivot the left-most element in the (sub)array you are working on.
- Illustrate the content of the whole array `a[]` after completing the second partitioning step. Use as pivot the left-most element in the (sub)array you are working on.

```
quicksort(a[], lo, hi):
      if (hi <= lo) return;

      j = partition(a, lo, hi);

      quicksort(a, lo, j-1);
      quicksort(a, j+1, hi);
```

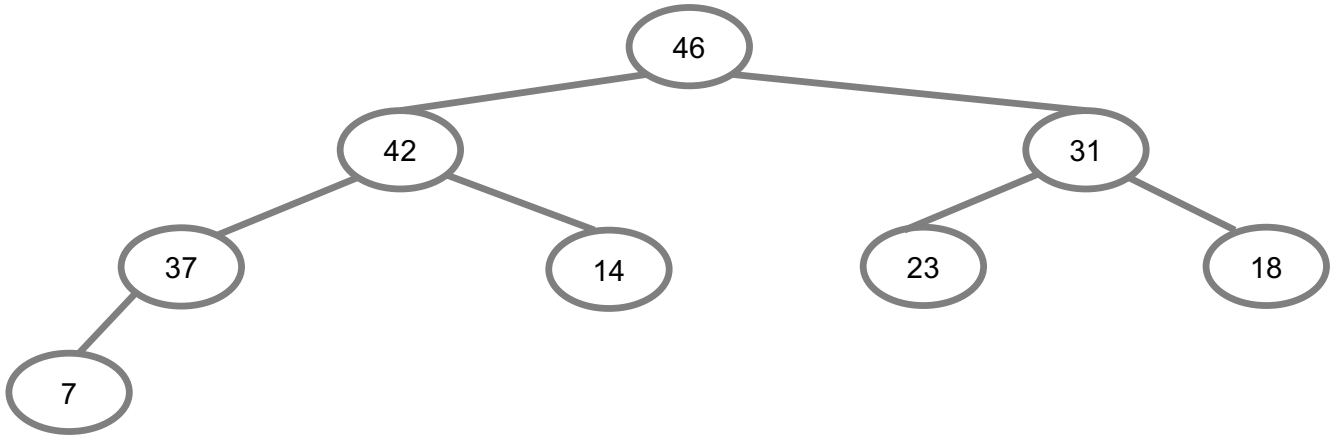
```
partition(a[], lo, hi):
      i = lo;
      j = hi+1;
      p = a[lo]
      while (true)
            while (a[++i] < p)
                if (i == hi) break;

            while (p < a[--j])
                if (j == lo) break;

            if (i >= j) break;
            swap(a[i], a[j]);

      swap(a[lo], a[j]);
      return j;
```

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Exercises
  - Quicksort
  - Heapsort

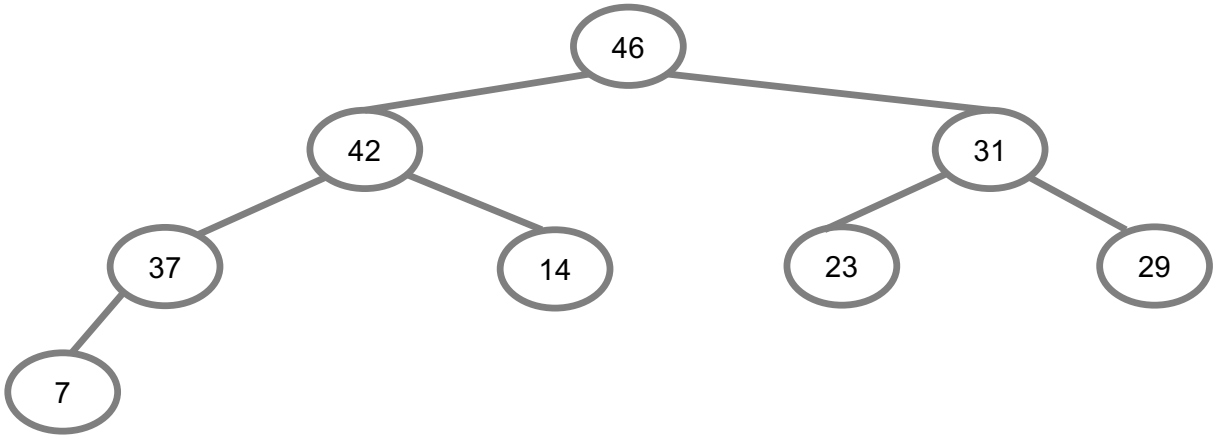
EXERCISES – HEAPSORT

Given the following tree-like representation of a max-heap:



- Show an array-like representation of the above max-heap. Leave the content of the array at index 0 empty, as seen in lectures.
- Illustrate step-by-step the changes that the array representation of the max-heap would undergo to insert key 50.
- Illustrate step-by-step the changes that the array representation of the max-heap would undergo to perform a delete-max operation.

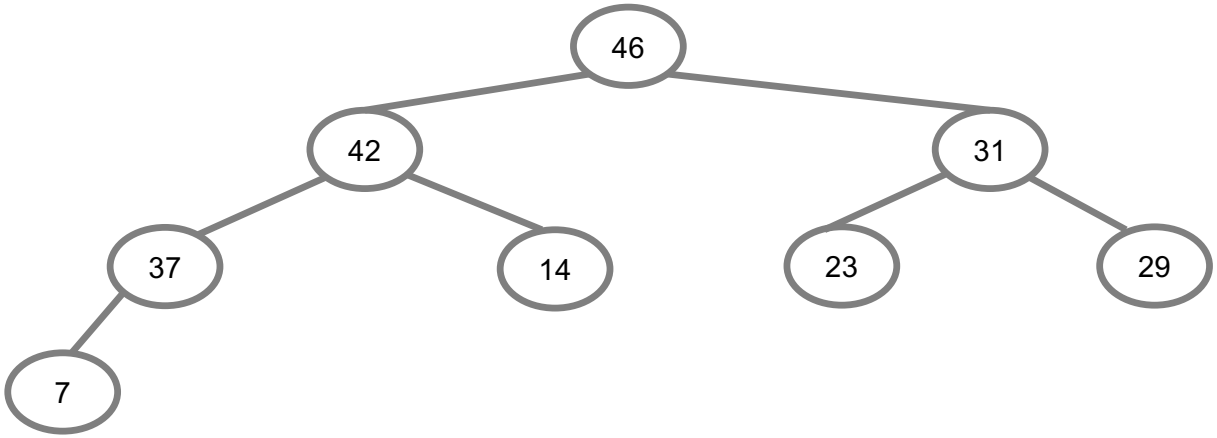
EXERCISES – HEAPSORT




```
insert(key):
    a[++N]=key
    swim(N)

swim(i):
    while (i > 1 && a[i/2]<a[i]):
          swap(a[i], a[i/2])
          i = i/2
```

[…] Illustrate step-by-step the changes that the array representation of the max-heap would undergo to insert key 50. *[…]*

EXERCISES – HEAPSORT



[…] Illustrate step-by-step the changes that the array representation of the max-heap would undergo to perform a delete-max operation. *[…]*

```
delMax():
    max=a[1];
    swap(a[1], a[N]);
    a[N--]=null;
    sink(1);
    return max

sink(i):
    while (2*i <= N):
        j = 2*i;
        if (j<N && a[j]<a[j+1]) j++;
        if (a[i]>=a[j]) break;
        swap(a[i], a[j]);
        i = j;
```

TAKE THE TEST …

Recap of topics 1 (Fundamentals) and 2 (Sorting)

https://tinyurl.com/yzhze56d